

Payment Gateway API Documentation

Integration guide for merchant developers — accept payments seamlessly with our secure, reliable Pay-In API.

CONTENTS

01 Authentication & Security

02 Initiate Payment

03 Check Transaction Status

04 Webhooks & Callbacks

05 Error Codes

Authentication & Security

All API requests must be authenticated using two mandatory headers. This ensures secure communication between your application and our servers.

Required Headers

Header	Description
X-API-Key	Your unique Merchant API Key (will be provided during onboarding)
X-Signature	HMAC-SHA256 signature generated for each request

Signature Generation

Every API request must include a cryptographic signature to verify its authenticity and integrity.

- Algorithm:** HMAC-SHA256
- Secret Key:** Your Merchant Secret Key (will be provided during onboarding)
- Logic:** Concatenate specific request fields with a pipe (|) separator, then hash the resulting string using your Secret Key.

Node.js Example

```
const crypto = require('crypto');

function generateSignature(fields, secretKey) {
  const signatureString = fields.join('|');
  return crypto
    .createHmac('sha256', secretKey)
    .update(signatureString)
    .digest('hex');
}

// Example usage for Initiate Payment
const signature = generateSignature(
  [merchantId, clientReferenceId, amount, customerEmail],
  'YOUR_SECRET_KEY'
);
```

Python Example

```
import hmac
import hashlib

def generate_signature(fields, secret_key):
    signature_string = '|'.join(str(f) for f in fields)
    return hmac.new(
        secret_key.encode(),
        signature_string.encode(),
        hashlib.sha256
    ).hexdigest()

# Example usage for Initiate Payment
signature = generate_signature(
    [merchant_id, client_reference_id, amount, customer_email],
    'YOUR_SECRET_KEY'
)
```

The specific fields used for signature generation vary by endpoint. Refer to each endpoint's documentation for the exact signature string logic.

Standard Response Format

All API responses follow a consistent JSON structure:

```
{
  "status": "SUCCESS",      // or "ERROR"
  "message": "Description of the result",
  "data": { ... }          // Response payload (varies by endpoint)
}
```

Initiate Payment

Creates a new payment session and returns a QR Code or Intent string that can be used to complete the payment.

POST /api/v1/payments/create

Headers

Header	Required	Description
Content-Type	Yes	application/json
X-API-Key	Yes	Your Merchant API Key
X-Signature	Yes	HMAC-SHA256 signature

Request Body

Field	Type	Required	Description
merchantId	String	Yes	Your unique Merchant ID
clientReferenceId	String	Yes	Your unique order/transaction reference (must be unique)
amount	Decimal	Yes	Payment amount (e.g., 1500.00)
customerName	String	Yes	Customer's full name
customerEmail	String	Yes	Customer's email address
customerPhone	String	Yes	Customer's phone number
currency	String	No	Pass value as INR
redirectUrl	String	No	URL to redirect after payment completion for Non H2H flow

Signature Generation

Signature String:
merchantId|clientReferenceId|amount|customerEmail

Note: The amount used in the signature string must exactly match the string format passed in the JSON payload, retaining its decimal precision. For example, if the JSON payload passes 1500.00, the signature string must use 1500.00 and not drop the decimals to 1500.

EXAMPLE REQUEST

```
curl -X POST https://api.xyz.com/api/v1/payments/create \
-H "Content-Type: application/json" \
-H "X-API-Key: your_api_key_here" \
-H "X-Signature: generated_signature_here" \
-d '{
  "merchantId": "MERCH123456",
  "clientReferenceId": "ORD-2026-00001",
  "amount": 1500.00,
  "customerName": "John Doe",
  "customerEmail": "john.doe@example.com",
  "customerPhone": "+919876543210",
  "redirectUrl": "https://yourwebsite.com/payment/redirect",
  "currency": "INR"
}'
```

SUCCESS RESPONSE

```
{
  "status": "SUCCESS",
  "message": "Payment session created successfully",
  "data": {
```

```
    "internalReferenceId": "PTX-20260122-ABC123",
    "qrCode": "...",
    "intentString": "upi://pay?pa=merchant@yourgateway&pn=MerchantName&am=1500.00",
    "redirectUrl": "https://pay.yourgateway.com/session/PTX-20260122-ABC123"
  }
}
```

Response Fields

FIELD	TYPE	DESCRIPTION
internalReferenceId	String	The gateway's unique reference ID for this transaction (save this for future references)
qrCode	String	Base64-encoded QR code image for UPI payment
intentString	String	UPI intent string for mobile app deep linking
redirectUrl	String	Hosted payment page URL (only for Non H2H flow)

Check Transaction Status

Retrieves the current status of a transaction in real-time. Use this endpoint to verify payment completion before fulfilling orders.

POST /api/v1/payments/status

Request Body

FIELD	TYPE	REQUIRED	DESCRIPTION
merchantId	String	Yes	Your unique Merchant ID
internalReferenceId	String	Yes	The gateway reference ID of the transaction

Signature Generation

Signature String:
merchantId|internalReferenceId

EXAMPLE REQUEST

```
curl -X POST https://api.xyz.com/api/v1/payments/status \
-H "Content-Type: application/json" \
-H "X-API-Key: your_api_key_here" \
-H "X-Signature: generated_signature_here" \
-d '{
  "merchantId": "MERCH123456",
  "internalReferenceId": "PTX-20260122-ABC123"
}'
```

SUCCESS RESPONSE

```
{
  "status": "SUCCESS",
  "message": "Transaction status retrieved successfully",
  "data": {
    "transactionStatus": "Approved",
    "internalReferenceId": "PTX-20260122-ABC123",
    "clientReferenceId": "ORD-2026-00001",
    "utr": "326145789012",
    "requestedAmount": 1500.00,
    "approvedAmount": 1500.00,
    "netAmount": 1485.00,
    "recipientUpiId": "gateway@upi",
    "customerUpiId": "customer@okaxis",
    "createdDate": "2026-01-22 14:30:00",
    "updatedAt": "2026-01-22 14:35:22",
    "customerEmail": "john.doe@example.com",
    "customerPhone": "+919876543210"
  }
}
```

Response Fields

FIELD	TYPE	DESCRIPTION
transactionStatus	String	Current status of the transaction (see values below)
internalReferenceId	String	The gateway's unique reference ID for this transaction
clientReferenceId	String	Your unique order/transaction reference
utr	String	Unique Transaction Reference
requestedAmount	Decimal	The originally requested payment amount
approvedAmount	Decimal	The amount that was approved
netAmount	Decimal	The settlement amount after fees/charges

recipientUpiId	String	The UPI ID that received the payment
customerUpiId	String	The UPI ID the customer paid from
createdDate	String	Transaction creation timestamp (format: yyyy-MM-dd HH:mm:ss)
updatedAt	String	Last status update timestamp (format: yyyy-MM-dd HH:mm:ss)
customerEmail	String	Customer's email address
customerPhone	String	Customer's phone number

Transaction Status Values

STATUS	DESCRIPTION
Pending	Payment initiated but not yet completed
Approved	Payment successfully completed and verified
LateApproved	Payment verified after the initial timeout window
Timeout	Payment session expired without completion
Failed	Payment failed or was declined

Webhooks & Callbacks

The system uses server-to-server webhooks to notify your system instantly when a transaction reaches a final state (e.g., Approved, LateApproved, or Failed). We will send an HTTP POST request to the Webhook URL configured in your merchant dashboard.

- Retry Policy:** If your server does not return a 200 OK response, the system will attempt to resend the webhook 5 times with exponential backoff delays (2s, 4s, 8s, 16s, 32s).
- Timeout:** Your server must acknowledge the request within 5 seconds to prevent a retry.

1. Webhook Payload

The payload is a JSON object containing the full transaction details and a security hash.

```
{
  "status": "SUCCESS",
  "message": "Transaction Status Updated",
  "data": {
    "transactionStatus": "Approved",
    "internalReferenceId": "PTX-20260122-ABC123",
    "clientReferenceId": "ORD-2026-00001",
    "utr": "326145789012",
    "requestedAmount": 1500.00,
    "approvedAmount": 1500.00,
    "netAmount": 1485.00,
    "recipientUpiId": "gateway@upi",
    "customerUpiId": "customer@okaxis",
    "createdDate": "2026-01-22 14:30:00",
    "updatedAt": "2026-01-22 14:35:22",
    "customerEmail": "john.doe@example.com",
    "customerPhone": "+919876543210"
  },
  "hash": "a1b2c3d4e5f6..." // HMAC-SHA256 Signature for verification
}
```

2. Verifying the Signature

To ensure the webhook is genuinely from the payment gateway and has not been tampered with, you must regenerate the hash on your server and compare it with the hash field received in the payload.

- Step 1:** Construct the signature string by concatenating the specific fields below in the exact order shown, separated by a pipe (|) character.
- Step 2:** Generate a hash of this string using the HMAC-SHA256 algorithm.
- Step 3:** Use your Merchant Secret Key as the cryptographic key for the HMAC process.
- Step 4:** Encode the result in Base64 and compare it to the received hash.

Important: The signature logic for Webhooks is different from the API Request signature. It strictly uses the order below.

Webhook Signature Pattern:
clientReferenceId|requestedAmount|transactionStatus

JAVA EXAMPLE

```
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;
import java.nio.charset.StandardCharsets;

// 1. Construct the data string
String dataToSign = payload.getData().getClientReferenceId()
    + "|" + payload.getData().getRequestedAmount()
    + "|" + payload.getData().getTransactionStatus();

try {
    // 2. Initialize HMAC-SHA256 with Secret Key
    Mac sha256_HMAC = Mac.getInstance("HmacSHA256");
```

```

    SecretKeySpec secret_key = new SecretKeySpec(MERCHANT_SECRET_KEY.getBytes(StandardCharsets.UTF_8), "HmacSHA256");
    sha256_HMAC.init(secret_key);

    // 3. Generate Base64 Signature
    String calculatedHash = Base64.getEncoder().encodeToString(sha256_HMAC.doFinal(dataToSign.getBytes()));

    // 4. Compare
    if (calculatedHash.equals(payload.getHash())) {
        response.setStatus(200); // Valid
    } else {
        response.setStatus(400); // Invalid
    }
} catch (Exception e) {
    e.printStackTrace();
}
}

```

NODE.JS EXAMPLE

```

const crypto = require('crypto');

// 1. Construct the data string
const data = req.body.data;
const dataToSign = `${data.clientReferenceId}|${data.requestedAmount}|${data.transactionStatus}`;

// 2. Generate HMAC-SHA256 Signature (Base64) using Secret Key
const calculatedHash = crypto
    .createHmac('sha256', MERCHANT_SECRET_KEY)
    .update(dataToSign)
    .digest('base64');

// 3. Compare
if (calculatedHash === req.body.hash) {
    res.status(200).send('OK');
} else {
    res.status(400).send('Invalid Signature');
}

```

PYTHON EXAMPLE

```

import hmac
import hashlib
import base64

# 1. Construct the data string
data = payload['data']
data_to_sign = f"{data['clientReferenceId']}|{data['requestedAmount']}|{data['transactionStatus']}"

# 2. Generate HMAC-SHA256 Signature (Base64) using Secret Key
calculated_hash = base64.b64encode(
    hmac.new(
        MERCHANT_SECRET_KEY.encode('utf-8'),
        data_to_sign.encode('utf-8'),
        hashlib.sha256
    ).digest()
).decode()

# 3. Compare
if calculated_hash == payload['hash']:
    return "OK", 200
else:
    return "Invalid Signature", 400

```


Error Codes

Standard HTTP status codes are used to indicate the success or failure of API requests.

200	OK	Request was successful. The response body contains the requested data.
400	Bad Request	Invalid request format or validation failed. Check the request body for missing or malformed fields.
401	Unauthorized	Authentication failed. Verify your X-API-Key and X-Signature are correct.
403	Forbidden	Access denied. Your IP may be blocked or your merchant account may be inactive. Contact support.
404	Not Found	The requested resource does not exist. Verify the internalReferenceId or endpoint URL.
409	Conflict	A conflict occurred. This typically means a duplicate clientReferenceId or UTR was submitted.
500	Internal Server Error	An unexpected error occurred on our servers. Please retry or contact support if the issue persists.

Error Response Example

```
{
  "status": "ERROR",
  "message": "Invalid signature. Please verify your signature generation logic.",
  "data": null
}
```